25.1 Canvas 概念

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~180k tokens，输出 ~5k tokens（本节）

前面的章节里，我们看了 OpenClaw 的 Agent 如何通过文字回复与用户交互、如何通过 Bash 工具执行命令、如何通过浏览器控制工具操纵网页。但这些交互方式有一个共同的局限——Agent 的输出本质上是**文本流**。当 Agent 需要向用户展示一张动态图表、一个交互式表单、甚至一个完整的小程序时，纯文本就力不从心了。

Canvas 系统正是为了解决这个问题而设计的。它为 Agent 提供了一个**可视化工作区**——一块可以渲染 HTML/CSS/JavaScript 的画布，Agent 可以向其中推送任意 Web 内容，用户可以在其中进行交互操作。

25.1.1 什么是 Canvas：Agent 驱动的可视化工作区

基本概念

Canvas 的核心思想很简单：**给 Agent 一块 WebView，让它可以自由渲染内容**。

在传统的聊天式 AI 交互中，Agent 的输出被限制在消息流里——文字、代码块、Markdown 表格。Canvas 提供了一个独立的渲染表面（rendering surface），Agent 可以：

1. **推送 HTML 页面**——Agent 生成 HTML/CSS/JS 文件，Canvas 服务器托管并渲染
2. **注入 JavaScript**——通过 `canvas.eval` 在运行中的页面执行任意 JS 代码
3. **截取快照**——通过 `canvas.snapshot` 获取当前渲染结果的截图
4. **控制显隐**——通过 `canvas.present` / `canvas.hide` 控制 Canvas 的可见性

这种设计使得 Agent 从一个"只会打字的助手"升级为"能操纵屏幕的助手"。

架构定位

Canvas 在 OpenClaw 的整体架构中处于**节点（Node）子系统**的一部分。回顾第 1 章的架构图：

```
用户 ←→ 原生客户端（macOS/iOS/Android）
                ↑
                │ WebSocket
                ↓
          Gateway 控制平面
                ↑
                │ Gateway 协议
                ↓
          Agent 运行时
                │
                ├── 文字回复（消息通道）
                ├── 工具调用（Bash / 浏览器 / Web 等）
                └── Canvas（可视化工作区）← 本章重点
```

Canvas 的数据流是这样的：

1. Agent 调用 `canvas` 工具（如 `canvas.present`）
2. 工具实现通过 Gateway 的 `node.invoke` 发送命令到目标节点
3. 目标节点（iOS/Android/macOS 原生应用）在本地 WebView 中执行命令
4. WebView 从 Canvas Host 服务器拉取 HTML 内容并渲染

```
Agent ──canvas.present──→ Gateway ──node.invoke──→ 原生应用
                                                      │
                                        Canvas Host ←──HTTP──┘
                                       （静态文件服务）     WebView 渲染
```

> **衍生解释——WebView**：WebView 是原生应用中嵌入的浏览器组件。在 iOS 上是 `WKWebView`（基于 WebKit 引擎），在 Android 上是 `android.webkit.WebView`（基于 Chromium），在 macOS 上同样是 `WKWebView`。它允许原生应用在界面中嵌入一个完整的网页渲染器，能执行 HTML/CSS/JavaScript。OpenClaw 利用 WebView 作为 Canvas 的渲染表面，这样 Agent 就可以通过 Web 技术来构建任意 UI。

Canvas Host：静态文件服务器

Canvas 的内容需要一个 HTTP 服务器来托管。OpenClaw 在 `src/canvas-host/server.ts` 中实现了这个服务器，称为 **Canvas Host**。它的职责很单纯：

职责	说明
静态文件服务	从 `~/.openclaw/canvas/` 目录提供 HTML/CSS/JS/图片等文件
路径安全	防止路径遍历攻击（`../` 不允许跳出根目录）
Live Reload	文件变更时通过 WebSocket 通知客户端自动刷新
A2UI 托管	同时托管 A2UI 渲染器的资源文件

Agent 的工作流程是：

1. 将 HTML 文件写入 `~/.openclaw/canvas/` 目录
2. 调用 `canvas.present` 让目标节点的 WebView 加载该文件
3. Canvas Host 提供文件服务，WebView 渲染内容

4. 文件变更时，Live Reload 自动刷新 WebView

这种"文件系统即 API"的设计非常简洁——Agent 不需要学习任何特殊的渲染协议，只需要会写 HTML 就可以驱动 Canvas。

Canvas 与浏览器控制的区别

你可能会问：第 16 章的浏览器控制不是也能操纵网页吗？Canvas 和它有什么区别？

维度	浏览器控制（第 16 章）	Canvas
控制对象	用户的桌面浏览器（Chrome/Edge/Brave）	嵌入在原生应用中的 WebView
内容来源	任意互联网网页	Agent 自己生成的 HTML
使用场景	网页自动化、爬虫、表单填写	Agent 驱动的可视化展示
协议	CDP / Playwright	Gateway node.invoke
平台	仅桌面（需要安装 Chrome）	macOS / iOS / Android

简言之：浏览器控制是"操纵别人的网页"，Canvas 是"渲染自己的网页"。

25.1.2 A2UI（Agent-to-UI）：AI Agent 直接操纵用户界面

从 HTML 推送到结构化 UI

Canvas 的基础模式——Agent 写 HTML，WebView 渲染——已经很强大了。但它有一个问题：**每次 UI 更新都需要重写整个 HTML 文件**。如果 Agent 只想在屏幕上追加一行文字、更新一个进度条，它不得不重新生成完整的 HTML 并触发页面刷新。

A2UI（Agent-to-UI）解决了这个问题。它是 OpenClaw 提出的一套**结构化 UI 协议**，允许 Agent 通过 JSONL（JSON Lines）命令流来增量地操纵用户界面。

> **衍生解释——A2UI**：A2UI 是 Agent-to-UI 的缩写，是 OpenClaw 原创的概念。它指的是 AI Agent 可以直接向用户推送结构化的 UI 描述（而非纯 HTML），由一个专用的渲染器（A2UI Renderer）解释并呈现为可交互的界面。可以把它理解为"AI 主导的前端渲染"——Agent 不仅能回复文字，还能展示按钮、表单、图表等交互式界面元素。

> **衍生解释——JSONL（JSON Lines）**：JSONL 是一种文本格式，每行一个 JSON 对象，行与行之间用换行符分隔。相比普通 JSON 数组，JSONL 的优势是**支持流式处理**——不需要等待整个数据完成，每读到一行就能解析一个对象。这对于 Agent 逐步构建 UI 非常合适：Agent 可以一条一条地发送 UI 指令，渲染器收到一条就执行一条。

A2UI 的工作方式

A2UI 的核心流程：

```
Agent ──a2ui_push(jsonl)──→ Gateway ──node.invoke──→ 原生应用
                                                        |
                                                   WebView 中的
                                                 <openclaw-a2ui-host>
                                                   自定义元素
                                                        |
                                                  a2ui.bundle.js
                                                   渲染器解析 JSONL
                                                   并更新 DOM
```

1. Agent 调用 `canvas` 工具的 `a2ui_push` 动作，附带 JSONL 数据
2. 命令通过 Gateway 转发到目标节点
3. 节点的 WebView 中运行着 A2UI 宿主页面（`index.html`）
4. 宿主页面中的 `<openclaw-a2ui-host>` 自定义元素接收 JSONL 并渲染

双向通信：用户动作回传

A2UI 不仅仅是单向推送。当用户在 A2UI 渲染的界面上点击按钮或提交表单时，**用户的操作可以回传给 Agent**。

这通过一个跨平台的"动作桥"（Action Bridge）实现。`injectCanvasLiveReload()` 函数会向每个 Canvas 页面注入一段 JavaScript，提供全局函数 `window.openclawSendUserAction()`：

```
// 注入的跨平台动作桥（简化版）
function postToNode(payload) {
  const raw = JSON.stringify(payload);

  // iOS 路径：通过 WKWebView 的 messageHandler
  const iosHandler = window.webkit?.messageHandlers?.openclawCanvasA2UIAction;
  if (iosHandler) {
    iosHandler.postMessage(raw);
    return true;
  }

  // Android 路径：通过 Android WebView 的 JS 接口
  const androidHandler = window.openclawCanvasA2UIAction;
  if (androidHandler) {
    androidHandler.postMessage(raw);
    return true;
  }

  return false;
}

// 全局暴露
window.OpenClaw = { postMessage: postToNode, sendUserAction: ... };
window.openclawSendUserAction = sendUserAction;
```

三个平台的消息路径：

平台	API	原理
iOS	`window.webkit.messageHandlers.openclawCanvasA2UIAction.postMessage()`	WKWebView 的 `WKScriptMessageHandler` 机制，允许 JS 向 Swift 发送消息
Android	`window.openclawCanvasA2UIAction.postMessage()`	Android WebView 的 `@JavascriptInterface` 注解，将 Java/Kotlin 对象暴露为 JS 全局对象
macOS	与 iOS 相同（`WKWebView`）	macOS 上的 `WKWebView` 与 iOS 共享相同的 WebKit API

> **衍生解释——JavaScript Bridge（JS 桥）**：在原生应用的 WebView 中，JavaScript 代码运行在一个沙箱环境中，默认无法访问原生功能（如摄像头、文件系统、通知等）。JS 桥是一种让 JavaScript 和原生代码互相调用的机制。在 iOS 上，Swift 通过 `WKScriptMessageHandler` 监听 JS 发来的消息；在 Android 上，Kotlin/Java 通过 `@JavascriptInterface` 注解将方法暴露给 JS。OpenClaw 利用这个机制实现了 A2UI 的用户动作回传。

A2UI 宿主页面

A2UI 的渲染发生在一个专用的宿主页面中（`src/canvas-host/a2ui/index.html`）。这个页面的结构包括三个层次：

```
┌──────────────────────────────────────┐
│ body（动画渐变背景）                   │
│  ├── <canvas>（底层 2D 画布，z-index: 1）│
│  ├── #openclaw-status（状态覆盖层，z: 3）│
│  └── <openclaw-a2ui-host>（A2UI 层，z: 4）│
└──────────────────────────────────────┘
```

层	元素	用途
底层画布	`<canvas id="openclaw-canvas">`	供低级 2D 绘图使用，自动适配设备像素比（DPR）
状态覆盖层	`#openclaw-status`	显示调试状态信息（默认隐藏，可通过 `?debugStatus=1` 启用）
A2UI 渲染层	`<openclaw-a2ui-host>`	承载 A2UI 渲染器输出的自定义元素，是用户看到的主要内容

宿主页面还暴露了一个全局 API `window.__openclaw`：

```
window.__openclaw = {
  canvas,              // <canvas> 元素引用
  ctx,                 // CanvasRenderingContext2D
  setStatus,           // 显示/隐藏状态卡片
  setDebugStatusEnabled, // 启用/禁用调试状态
};
```

平台自适应

A2UI 宿主页面通过 CSS 自定义属性和平台检测实现了跨平台适配：

```
/* A2UI 自定义元素的 CSS 自定义属性 */
openclaw-a2ui-host {
  --openclaw-a2ui-inset-top: 28px;      /* 顶部安全区域 */
  --openclaw-a2ui-inset-right: 0px;
  --openclaw-a2ui-inset-bottom: 0px;
  --openclaw-a2ui-inset-left: 0px;
  --openclaw-a2ui-scroll-pad-bottom: 0px;
}
```

这些自定义属性对应移动设备的安全区域（Safe Area），原生应用在加载 WebView 时会将实际的安全区域值注入这些属性，确保 A2UI 内容不会被刘海屏、底部手势条等遮挡。

平台检测在页面加载时执行：

```
// 平台检测（简化版）
const params = new URLSearchParams(window.location.search);
const platform = params.get("platform");  // 优先使用 URL 参数
if (platform) {
  document.documentElement.dataset.platform = platform;
} else if (/android/i.test(navigator.userAgent)) {
  document.documentElement.dataset.platform = "android";
}
```

Android 平台会使用更高不透明度的渐变背景，以适配 OLED 屏幕的显示特性：

```
/* Android 特定样式：更高对比度的渐变 */
:root[data-platform="android"] body {
  background:
    radial-gradient(... rgba(42, 113, 255, 0.62) ...),  /* iOS: 0.18 */
    radial-gradient(... rgba(255, 0, 138, 0.52) ...),   /* iOS: 0.14 */
    #0b1328;                                             /* iOS: #000 */
}
```

Canvas 与 A2UI 的关系

最后，厘清 Canvas 和 A2UI 的关系：

```
Canvas（广义）
├── Canvas 基础模式
│   └── Agent 写 HTML → Canvas Host 托管 → WebView 渲染
│       适用于：完整的 Web 应用、自定义页面
│
└── A2UI 模式
    └── Agent 发 JSONL → 渲染器解析 → 增量更新 DOM
        适用于：结构化 UI、实时更新、对话式界面

两种模式共享同一个 Canvas Host 服务器和 WebView 容器。
```

Canvas 是底层基础设施（WebView + 静态文件服务），A2UI 是基于 Canvas 的高级抽象（结构化 UI 协议）。Agent 可以根据需要选择：

- 需要完全自定义的 UI → 使用 Canvas 基础模式，直接推送 HTML
- 需要快速构建标准化界面 → 使用 A2UI 模式，发送 JSONL 命令

本节小结

1. **Canvas** 是 OpenClaw 的可视化工作区，让 Agent 从"只会打字"升级为"能渲染界面"
2. Canvas 的核心是一个**静态文件服务器**（Canvas Host），Agent 将 HTML 文件写入指定目录，由 WebView 加载渲染
3. Canvas 通过 Gateway 的 `node.invoke` 机制将命令转发到原生应用节点，节点在本地 WebView 中执行
4. **A2UI**（Agent-to-UI）是基于 Canvas 的高级协议，通过 JSONL 命令流实现增量式 UI 更新
5. A2UI 通过**跨平台动作桥**（JS Bridge）实现双向通信——iOS 用 `WKScriptMessageHandler`，Android 用 `@JavascriptInterface`
6. A2UI 宿主页面包含三个层次：底层 Canvas 画布、状态覆盖层、A2UI 渲染层
7. 平台自适应通过 CSS 自定义属性和 `data-platform` 数据集实现，Android 有特定的高对比度样式